ADDING PRODUCT FEEDS TO YOUR EXPERIENCE

TABLE OF CONTENTS

Adding Product Feeds to Your Experience

Last Updated: 06/01/2022

Use the Product Feeds API **to show relevant Nike product-related content, including details about the products with images, videos, and more**.

> **TIP:** Before using this guide, you should have completed Using Nike APIs and Product Feeds Overview.

Introduction

The Product Feeds API provides product data and content in the form of Cards, Threads, and Feeds.

What are Cards, Threads, and Feeds?

- **Cards** are portions of a Thread, such as for use in displaying a heading or banner
- **Threads** contain Nike product information or content that tells a Nike story
- Multiple Threads can be displayed in **Feeds**, customized for your users based on their chosen preferences in a Nike experience

> **TIP**: Card data is also available from the Product Feed Cards v2 API.

What Product Data and Content are Available?

The Product Feeds API combines data from many Nike Cloud APIs. To understand the variety of data available, and the sources of data, use the following table:

Table 1: Types of Data Available from Product Feeds

Cloud API	Data Type	Examples	Data Source
Merchandised Product Information	Product attributes per style-color, country	styleCode, colorCode, status, genders, sportTags	Prodigy & Product Information Services
Merchandised Product Price	Pricing per style-color, country	msrp, currentPrice, discounted, currency	Prodigy & Product Information Services
Merchandised Product SKU	SKU-related attributes per style-color-size, country	gtin, nikeSize, localizedSize, commodityCode	Prodigy & Product Information Services
Merchandised Product Content	Display-oriented content by style, country	colorDescription, fullTitle, colors, bestFor, athletes, widths	Prodigy & Product Information Services
Product Inventory Availability	Inventory availability by style-color	true/false	Sterling
GTIN Inventory Availability	Inventory availability by GTIN	true/false	Sterling
Published Content	Authored cards and threads	title, seo slug, image URL, video URL, text	Nike CMS (Content Management System)
Launch Views	Launch attributes by style-color (SNKRS, Bootroom only)	method, startEntryDate, stopEntryDate	Launch Admin Tool
Customized PreBuilds	Customized prebuild (e.g. suggested NikeID shoe design)	designId, status, merchGroup	Consumer experiences, Prodigy

> **TIP:** Another great way to evaluate the types of data returned is to analyze a sample API response.

What are Channels and Why Do I Need One?

A channel is a distinct experience where Nike products are showcased and made available for purchase, e.g. SNKRS, Nike.com, Nike Retail. Each channel has a unique `channelId` (channel identifier), which along with `language` and `marketplace`, allows you to get the appropriate data for your experience.

Accessing Retail Products

Both Retail and Digital products are available via Product Feeds. Here are some important considerations for working with Retail data:

- Use the Nike.com channelId, not the Retail channelId (except Nike Assist app)
- Use one or more of the following values in the `?filter` parameter:

Table 2: Retail location-related query parameters for Product Feeds

Param	Description	Example
`locationIds`	Location Id from Available GTINs API	filter=locationIds(7B988866-4C8C-4062-AF86-E26C14A4AE96)
`locationAvailabilityMethods`	Availability methods ie. SHIP, INSTORE, PICKUP	filter=locationAvailabilityMethods(INSTORE)
`locationAvailabilitySizes`	Sizes available in the selected location	filter=locationAvailabilitySizes(4a528b33-a71b-342c-baa2-616a89985fef)

Sample URL:

```
https://api.nike.com/product_feed/threads/
v3?filter=marketplace(US)&filter=language(en)&filter=channelId(d9a5bc42-4b9c-4976-858a-
f159cf99c647)&anchor=0&count=20&sort=lastFetchTimeDesc&searchTerms=pegasus&filter=locationIds(C5E2DB2B-
DBD5-41FE-91CE-91D774F1E9D5)&filter=locationAvailabilityMethods(INSTORE)
```

> **TIP:** A Feed can be associated with one or more channels, opening up the personalized Feed to many Nike experiences.

V2/V3 Differences

Product Feeds v3 is an extension of Product Feeds v2 that was created to handle differences in external vs internal traffic.

The differences between v2 and v3 are as follows:

- Product Feeds v3 does not support data from availableSkus in the response, instead replacing it with `productInfo.availableGtins`. (See API docs for schema at Available GTINs V3)
- `productInfo.imageUrls`: The imageUrls section is deprecated. Images should be used from publishedContent.
- Product Feeds v3 has different endpoints for internal calls with valid JWT authorization. The endpoints for internal JWT end in `/secured`.
- If Product Feeds v3 is called with an authorized JWT on the `/secured` endpoints, it will allow the same query parameters and filters as Product Feeds v2.
- If Product Feeds v3 calls are made to the non-secured endpoints, the following restrictions apply:
  - No fields param will be allowed, and a 400 error will be returned on the multi GET or by id GET endpoints.
  - On the List GET endpoint, the filter param can only contain a single value. No filter arrays will be supported, except in the exclusiveAccess filter.
  - On the List GET endpoint, the count param can only contain the values of 50 or 100
  - On the List GET endpoint, the anchor param must be a multiple of 50.

Get a List of Product Threads

List all Product Threads for a channel, language, marketplace, feed ID, SEO slug, style-color, gender, keywords, and more

Now you know what Cards, Threads, and Feeds are, but how do you get them and use them?

To get a list of Threads, execute a request to the Threads List endpoint, including at minimum the required filter query parameters for **channelId**, **language**, and **marketplace**.

Sample Threads List request URI:

```
https://api.nike.com/product_feed/threads/v3?filter=channelId(d9a5bc42-4b9c-4976-858a-f159cf99c647)&filter=language(en)&filter=marketplace(US)
```

> **TIPS:**
>
> - For a sample *Threads List* response, see here.

How to Get Only the Threads You Need

If there are more threads in the API response than you want, add more specific identifiers to the request as follows.

1. **Filters**

   Use any of the supported **filter** query parameters to get more specific results in the response. For example, to only get content for a specific Nike style-color code, append a query parameter like `filter=publishedContent.properties.products.styleColor(942198-700)` to the request URI.

2. **Sorting**

   Sort the results using any of the supported **sort** query parameters. For example, to sort by the current price in ascending order, append query parameter `sort=productInfo.merchPrice.currentPriceAsc` to the request URI.

3. **Fields**

   Request only the fields that you want in the response by using the **fields** query parameter. For example, to return fields threadId, styleColor, and MSRP, append query parameter `fields=id,channelId,productInfo.merchProduct.styleColor,productInfo.merchPrice.msrp` to the request URI.

> **TIPS:**
>
> - The Product Feeds API Reference is the source of truth for all supported query parameters.
> - The Product Feeds API restricts each response to 10,000 products for performance reasons. If you need >10k products at a time, reach out to the Product Owner about using the Product Feeds Stream API.

Can I Use Product Feeds to Search for Products?

You can use the Product Feeds API for basic product search by including the **searchTerms** query parameter. The API attempts to match the string sent in searchTerms to a fixed set of fields in the product data (see Product Feeds API Reference for the list of fields).

However, for a more accurate search, or if you are merchandising a product wall, it's strongly recommended that you use the Rollup Threads API. Rollup Threads can be used in conjunction with Smart Search rules to influence the search results based on your specific use case.

See Adding Rollup Threads to Your Experience and Understanding Search Results for more.

Terminology Differences Between CMS and Product Feeds

The Product Feeds API includes product content from Nike CMS in responses. Nike CMS sometimes uses field names that are different from Product Feeds. For example, the CMS **collectionGroupId** that you will see in responses is the same as the **channelId** query parameter you might send to Product Feeds.

Here is a terminology guide between Nike CMS and Product Feeds:

Table 3: Common CMS, Product Feeds Terms

CMS Term	Product Feeds Term
Collection Group	Channel
Collection	Feed
Content/Thread	Thread
Node	Card

What is the Customized PreBuild Section of the Response?

If you are getting data in the **objects.productInfo.customizedPreBuild** section of the response, then one of the threads that you've requested contains a customizable prebuild product.

A prebuild is a design for a customizable (e.g., NIKEiD) product invented by merchandisers/designers to demonstrate how consumers can personalize the product. These "inspiration" designs are merchandised within specific experiences and can be found on product walls, product display pages and in marketing materials.

You will be able to identify the presence of prebuilds when **objects.publishedContent.properties.threadType** field contains the value **nikeid_soldier**.

Get a Product Thread by ID

Get a Specific Product Thread by its ID

Get a single product thread by its unique identifier by executing a request to the Thread by ID endpoint.

Sample Thread by ID request URI:

```
https://api.nike.com/product_feed/threads/v3/bcbeae50-28a5-404d-9941-fbbdff0c7860
```

API Quick Reference

Product Feeds v3 Endpoints

Table 4: Product Feeds Endpoints

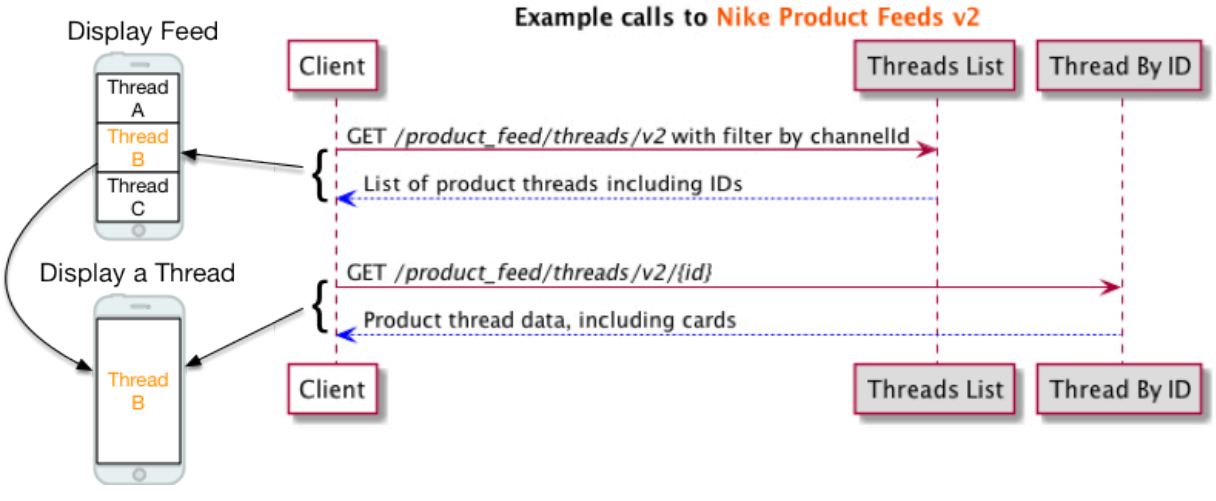
HTTP Verb	Endpoint Name	Endpoint Description	URI Format
GET	Product Threads List	Get all threads for a channel, marketplace, language combination	`/product_feed/threads/v3{?filter,fields,anchor,count,sort,searchTerms}`
GET	Product Thread by ID	Get a specific thread by its identifier	`/product_feed/threads/v3/{id}{?channel,marketplace,language,fields,preview}`

Best Practices

Listed below are some best practices for working with Product Feeds.

Example Implementation Diagram

Here is an example of a sequence of API calls to get content from Product Feeds v2 (note: v3 follows the same pattern):



Test Environment

Product Feeds v3 has a test environment available at host https://experience.test.commerce.nikecloud.com.

Apart from the host, you can use the same URL, for example: `https://experience.test.commerce.nikecloud.com/product_feed/threads/v3?filter=marketplace(US)&filter=language(en)&filter=channelId(d9a5bc42-4b9c-4976-858a-f159cf99c647)`.

Some considerations about using the test environment:

- The functionality of the API is the same in test vs. production, except new features that are not yet deployed to production.
- Test has a different set of product data than production, but the test data set is similar. For example, many of the constants are the same in test, i.e. the marketplace, language, and channelId, as they are in production.
- Inventory availability data is scarce in the test environment. At the product level, i.e. the values in `productInfo.availability.available`, might show as 'false' in test most of the time. Availability data at the SKU/size level is *not* present in test.
- All performance testing activities should be done in test and not in production.

Troubleshooting

Listed below are ways to troubleshoot unexpected responses using this API.

Use Troubleshooting Tools

- Use the general troubleshooting tips in the Using Nike APIs guide.
- Use a Splunk report (requires access) found here to check for issues with your request.
- Contact the Product Feeds Team on the #nde-product-feeds Slack channel for assistance.

Common Questions

Who do I contact with questions about what I'm seeing in the `productInfo` or `publishedContent` sections of the response?

- The data in `productInfo` and `publishedContent` is not owned by the Product Feeds team. Refer to Thread Response Ownership Breakdown to find the Slack channel of the team responsible for that data.

How do I know what product attributes are available for me to use to request Threads?

- Unless you are doing a keyword search using the `searchTerms` query parameter, you need to know in advance which product attributes to include in your requests. The source of product attributes (e.g. slugs, 'best for', and other attributes) is Nike's Prodigy system.

Why isn't my feed showing up?

- The feed may have failed validation and was marked inactive. Only active threads with a valid publish date will be returned by this API. Contact the Product Feeds Team on the #nde-product-feeds Slack channel to check if the feed failed validation and why.
- The feed might not yet be published. It can take up to 15 minutes to publish a change from AEM and have it be reflected in the Feeds API.

Why am I getting an empty 200 response from *Threads List*?

- There might not be any threads that meet the criteria that you specified, particularly when using optional filters. For example, if you send a request with `filter=productInfo.merchProduct.styleCode(999999)` and there are no threads for style code 999999, you will get an empty 200 response.
- The `publishStartDate` for the requested threads might be in the future, or conversely the `publishEndDate` might be in the past. In other words, if today's date is not within the publish start/end range for the thread, the thread will not be returned in the response.
- The `softLaunchDate` on the product is in the future. The softLaunchDate, when present, overrides the publishStartDate, and thus can affect whether the thread is returned or not.

Why am I getting a 404 error from Thread by ID when I know that the thread ID is valid?

- Today's date might be outside the publish start/end range for the thread.
- The `catalogId` on the product might be blank. To troubleshoot, send a request to the Merchandised Products API with the affected product ID (e.g. https://api.nike.com/merch/products/v2/c98f12d7-7dee-5775-b4a6-c83d0d2dcb9a) to see if a catalog ID is present or not. If not, that is the reason that the thread is not being returned.

> **TIP:** Be careful not to confuse `legacyCatalogId`, which like `catalogId` is also present in the threads response under `productInfo.merchProduct`, but does not affect thread visibility.

Terms of Service

It is highly recommended that you send a caller ID header in every request to this API to help troubleshoot unexpected responses. See the Registration section of the Using Nike APIs guide on how to create and register your caller ID.

Authentication

There are no authentication requirements for Product Feeds except when using the `preview` query parameter to preview a Feed or Thread, which is not common.

Contacting the Team

Need to contact the Product Feeds team?

Slack	#nde-product-feeds
Confluence Space	Product and Feeds API
Team Contacts	Andy Sun

Document Change Log

Summary	Date
Initial publish	01/23/2018
Referenced new Retail data	07/06/2020
Updated for v3	10/20/2021

Related Links

- Using Nike APIs
- Glossary